

SWEN90010 - High Integrity Systems Engineering

Advanced Specification: Ghost Code

Christine Rizkallah



christine.rizkallah@unimelb.edu.au



<http://people.eng.unimelb.edu.au/rizkallahc>

Ghost Code: additional code whose only purpose is to help prove the original program correct

Can be used **only in specifications**, i.e. in pre/postconditions, loop invariants, Assert pragmas, etc.

Common uses:

to express properties you want to prove or assume

to keep track of information in your program that your specifications need to talk about (e.g. history, results of intermediate computations)



```
package SimpleStack with SPARK_Mode is
  type SimpleStack is private;
  function Size(S : in SimpleStack) return Integer;
  function Storage(S : in SimpleStack; Pos : in Integer) return Item with
    Pre => (Pos >= 1 and Pos <= Max_Size);
  procedure Init(S : out SimpleStack) with
    Post => (Size(S) = 0);
  procedure Push(S : in out SimpleStack; I : in Item) with
    Pre => (Size(S) /= Max_Size),
    Post => (Size(S) = Size(S'Old) + 1 and
      Storage(S,Size(S)) = I and
      (for all J in 1..Size(S'Old) => Storage(S,J) = Storage(S'Old,J)));
```

A prime candidate that we have already seen



```
package SimpleStack with SPARK_Mode is
  type SimpleStack is private;
  function Size(S : in SimpleStack) return Integer;
  function Storage(S : in SimpleStack; Pos : in Integer) return Item with
    Ghost,
    Pre => (Pos >= 1 and Pos <= Max_Size);
  procedure Init(S : out SimpleStack) with
    Post => (Size(S) = 0);
  procedure Push(S : in out SimpleStack; I : in Item) with
    Pre => (Size(S) /= Max_Size),
    Post => (Size(S) = Size(S'Old) + 1 and
      Storage(S,Size(S)) = I and
      (for all J in 1..Size(S'Old) => Storage(S,J) = Storage(S'Old,J)));
```

Prevents **Storage** from being called in real code
(it was never meant to be, anyway)

EXAMPLE 2: MATHEMATICAL INTEGERS

A trivial package

```
package Integer_Operations is
    function Add(X : Integer, Y : Integer) return Integer;
    function Subtract(X : Integer, Y : Integer) return Integer;
    -- etc ..
end Integer_Operations;
```

How should “Add” behave?



THE UNIVERSITY OF
MELBOURNE

Add should behave like addition on mathematical integers

Add(x,y) should give the mathematical $x + y$ whenever the mathematical value of $x + y$ is between Integer'First and Integer'Last (inclusive)

What is wrong with this specification?

```
function Add(X : Integer; Y : Integer) return Integer with  
  Pre => (X + Y >= Integer'First and X + Y <= Integer'Last);
```

Mathematical Integers in SPARK Ada

Not directly supported in GNAT Prover before 2020

Can be used as **Ghost Code** in GNAT 2019 via a custom library (<https://github.com/joffreyhuguet/curve25519-spark2014>)

The package is called **Big_Integers**
type is called **Big_Integer**

Need to explicitly convert between **Integer** types and
Big_Integers
(To_Big_Integer, To_Integer, etc.)

The “Add” specification

```
function Add(X : Integer; Y : Integer) return Integer with
  Pre => (In_Range(Arg => To_Big_Integer(X) + To_Big_Integer(Y),
                  Low => To_Big_Integer(Integer'First),
                  High => To_Big_Integer(Integer'Last))),
  Post => Add'Result = To_Integer(To_Big_Integer(X) + To_Big_Integer(Y));
```

In_Range(A,L,H) abbreviates $L \leq A$ and $A \leq H$

Postcondition: convert X to a mathematical integer; do likewise for Y. Add them together (mathematical addition). Then the result of Add(X,Y) is equal to that integer (converted back to a fixed-width machine integer)



EXAMPLE 3: SUMMING AN ARRAY

Summing an Integer Array

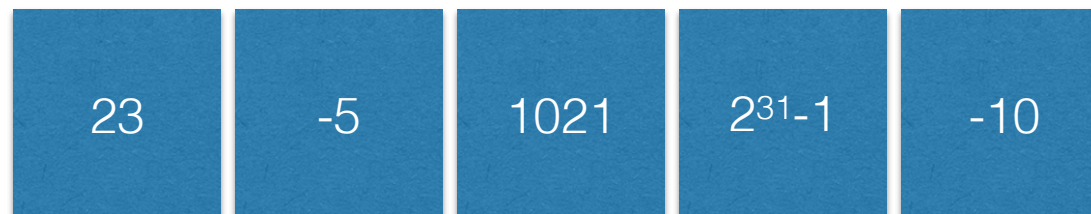


THE UNIVERSITY OF
MELBOURNE

```
type Integer_Array is array(Positive range <>) of Integer;
```

```
function Sum_Blind(A : Integer_Array) return Integer
```

Assume we blindly sum from left to right on a 32-bit machine



Summing these elements will overflow on a 32-bit machine

Specifying Absence of Overflow

How should **Sum_Blind** behave?

Assuming absence of overflow **throughout the entire summation**, it should behave like mathematical summation

Requires talking about absence of overflow at all **intermediate points of the computation**

Specifying Absence of Overflow

Solution: build a **ghost** array of mathematical integers that are the **partial sums**

A	23	-5	1021	$2^{31}-1$
Sum_Acc	23	$23+-5$	$23+-5+1021$	$23+-5+1021+2^{31}-1$

Precondition: all values in **Sum_Acc** are between Integer'First and Integer'Last

Postcondition: The answer is equal to the final value in **Sum_Acc**, i.e. to **Sum_Acc(Sum_Acc'Last)**

Specifying Absence of Overflow



Solution: we write a **ghost function** to compute the array of partial sums

```
type Partial_Sum is array(Positive range <=>) of Big_Integer with Ghost;

function Sum_Acc(A : Integer_Array) return Partial_Sum with
  Ghost,
  Pre => A'Length > 0,
  Post => Sum_Acc'Result'Length = A'Length and then
           Sum_Acc'Result'First = A'First and then
  (for all J in A'Range =>
    Sum_Acc'Result(J) = (if J=A'First then To_Big_Integer(A(A'First)) else
                        Sum_Acc'Result(J-1) + To_Big_Integer(A(J))));
```

```
function Sum_Blind(A : Integer_Array) return Integer with
  Pre => (if A'Length > 0 then (for all J in Sum_Acc(A)'Range =>
    In_Range(Sum_Acc(A)(J), To_Big_Integer(Integer'First),
              To_Big_Integer(Integer'Last)))
    else True),
  Post => Sum_Blind'Result = (if A'Length = 0 then 0 else
    To_Integer(Sum_Acc(A)(A'Last)));
```